Lab 8. Model Inference with RPi

Objective

- Learn about the ML model deployment into edge device like Raspberry Pi (RPi).

- Learn about ONNX for inference the deployed model

- Learn about AI accelerator hardware NPU

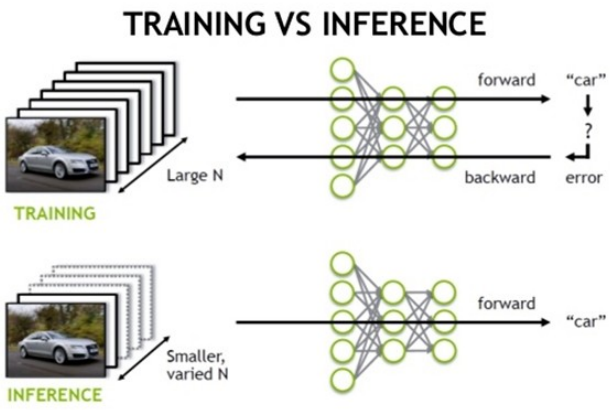
What is AI Inference?

AI inference involves **applying a trained machine learning model to make predictions** or decisions based on new, unseen data. This phase contrasts with the training period, where a model learns from a dataset by adjusting its parameters (weights and biases) to minimize errors, preparing it for real-world applications.

Inference vs. Training

Understanding the distinction between training and inference phases in AI is crucial for leveraging AI technologies effectively:

- **Objective**: The objective of training is to develop a model that accurately represents the underlying patterns of the training data, while inference uses this model to make predictions on new data.
- **Computational Requirements**: Training is computationally intensive as it involves processing large datasets and continuously adjusting the model's parameters. Inference, however, is generally less demanding since it involves applying the already learned parameters to new data.
- **Duration**: The training process can be lengthy, taking hours to weeks depending on the complexity and volume of the data. In contrast, **inference can be executed rapidly**, often in milliseconds, to provide **real-time** insights.



Experiment 1. Camera Interfacing in Raspberry Pi 5.



Step 1. Test the camera

On the Raspberry Pi 5, run this command to test the camera by using the app from raspberry pi:

```
$ rpicam-hello -t 10s
```

Step 2. Activate the venv python environment for ONNX and OpenCV

We have already created the python virtual environment with **venv**, which consists of these packages:

```
numpy
pillow
opencv-python
onnx
onnxscript
onnxruntime
```

To activate the environment:

$ cd ~/venv

$ source ml_onnx/bin/activate

Step 3. Interface the camera with OpenCV-python

Create a simple Python program (.py) by using these 2 example codes for interfacing the camera with the camera. One for the basic interface and another one for using the drop-frame technique to fix the buffer issue.

Try to run both programs. But, we will focus on using the drop-frame technique later in the next section.

capture_basic.py

```
"""
capture_basic_rpicam3.py

Basic camera test for RPi Camera Module 3 (Picamera2) using the
producer / consumer pattern.

    [camera] --producer thread--> [queue (latest frame only)] --consumer--> [process + display]

- Producer (thread)   : grabs frames from the camera as fast as the sensor delivers them.
- Consumer (main)     : takes the most recent frame, runs a blocking process(), displays it.
                        OpenCV GUI calls (imshow / waitKey) are kept in the main thread
                        because most HighGUI backends (Qt / GTK) are not thread-safe.

Because the queue holds only one frame, a slow process() never builds up latency:
old frames are dropped and the consumer always works on the freshest image.
Replace process() with your own pre-processing + Hailo inference later.

Press 'q' (or Ctrl+C) to quit.
"""
import argparse
import queue
import threading
import time

import cv2
from picamera2 import Picamera2
from libcamera import controls


def process(frame):
    # some blocking process (simulates pre-processing + model inference)
    time.sleep(0.2)
    return frame


def put_latest(q, item):
    """Put item into a size-1 queue, dropping the old item if present.
    Returns True if an old frame was dropped."""
    dropped = False
    try:
        q.put_nowait(item)
    except queue.Full:
        try:
            q.get_nowait()
            dropped = True
        except queue.Empty:
            pass
        q.put_nowait(item)
    return dropped
```

```python
def task_inputframe(picam2, q, stop_event, stats):
    """Producer: read frames from the camera and push them to the queue."""
    frame_id = 0
    t_prev = time.perf_counter()
    try:
        while not stop_event.is_set():
            # capture_array() blocks until the next frame and returns a copy,
            # so it is safe to hand over to another thread.
            frame = picam2.capture_array("main")
            frame_id += 1

            if put_latest(q, (frame_id, frame)):
                stats["dropped"] += 1

            t_now = time.perf_counter()
            stats["capture_fps"] = 0.9 * stats["capture_fps"] + 0.1 / max(t_now - t_prev, 1e-6)
            t_prev = t_now
    except Exception as e:
        if not stop_event.is_set():   # errors while closing the camera are expected
            print("capture error:", e)
    finally:
        stop_event.set()


def draw_info(frame, frame_id, stats, process_fps):
    text = [
        f"frame #{frame_id}",
        f"capture fps: {stats['capture_fps']:.1f}",
        f"process fps: {process_fps:.1f}",
        f"dropped    : {stats['dropped']}",
    ]
    for i, t in enumerate(text):
        cv2.putText(frame, t, (10, 30 + i * 30), cv2.FONT_HERSHEY_SIMPLEX,
                    0.8, (0, 255, 255), 2, cv2.LINE_AA)




def main():
    parser = argparse.ArgumentParser(description="RPi Camera Module 3 producer/consumer test")
    parser.add_argument("--width", type=int, default=1280)
    parser.add_argument("--height", type=int, default=720)
    parser.add_argument("--fps", type=float, default=30.0)
    args = parser.parse_args()

    picam2 = Picamera2()
    # Video configuration (not still configuration) is meant for continuous streaming:
    # several buffers, fixed frame rate, fast capture.
    # Note: Picamera2 "RGB888" is stored in B,G,R byte order -> already what OpenCV expects.
    config = picam2.create_video_configuration(
        main={"size": (args.width, args.height), "format": "RGB888"},
        controls={"FrameRate": args.fps},
    )
    picam2.configure(config)

    # Camera Module 3 has autofocus: enable continuous AF if the sensor supports it
    if "AfMode" in picam2.camera_controls:
        picam2.set_controls({"AfMode": controls.AfModeEnum.Continuous})

    q = queue.Queue(maxsize=1)
    stop_event = threading.Event()
    stats = {"capture_fps": 0.0, "dropped": 0}

    picam2.start()
    t1 = threading.Thread(target=task_inputframe, args=(picam2, q, stop_event, stats), daemon=True)
    t1.start()
```

```
    print("start capture")
    process_fps = 0.0
    t_prev = time.perf_counter()
    try:
        while not stop_event.is_set():
            # Consumer: wait for a frame (timeout so we can still react to stop_event)
            try:
                frame_id, frame = q.get(timeout=0.5)
            except queue.Empty:
                continue

            # some blocking process
            out = process(frame)

            t_now = time.perf_counter()
            process_fps = 0.9 * process_fps + 0.1 / max(t_now - t_prev, 1e-6)
            t_prev = t_now

            # output / display
            draw_info(out, frame_id, stats, process_fps)
            cv2.imshow("output", out)

            # check keyboard for exit program
            key = cv2.waitKey(1) & 0xFF
            if key == ord('q'):
                break
    except KeyboardInterrupt:
        pass
    finally:
        stop_event.set()
        t1.join(timeout=2.0)
        picam2.stop()
        picam2.close()
        cv2.destroyAllWindows()
        print("capture is over")


if __name__ == "__main__":
    main()
```

Experiment 2. Deploy the ONNX model in Raspberry Pi 5.

Create a Python program by using this example code (capture_mnist.py) for the MNIST real-time classification on the raspberry pi 5. ONNX runtime (python version) will be used to run our ONNX model.

The ONNX can be used in the python with **onnxruntime** module as shown here:

```
import onnxruntime

# Load the ONNX model
session = onnxruntime.InferenceSession('model_mnist.onnx')
```

```
# Prediction
predict = session.run(None, {'input': data_input})
```

Create **inference_mnist_onnx.py** program to do the inference using onnx runtime:

inference_mnist_onnx.py

```
"""
inference_mnist_onnx.py

Run the MNIST digit classifier (DIY ANN) with ONNX Runtime on the Raspberry Pi 5 CPU.

    python3 inference_mnist_onnx.py --mode image                    # test images img/0.jpg ... 9.jpg
    python3 inference_mnist_onnx.py --mode camera                   # Pi Camera Module 3, live FPS
    python3 inference_mnist_onnx.py --mode camera --camera usb      # USB webcam
    python3 inference_mnist_onnx.py --mode camera -r                # camera mounted 90 deg counter clockwise

Press 'q' in the video window or Ctrl+C to quit.
Compare with inference_mnist_hailo.py (same program on the Hailo-8 NPU):
the "inference ... ms" value is the speed of the model itself. The FPS is the speed of the whole loop
and is limited by the camera (30 FPS) and the display, so it is about the same for both.
"""
import argparse
import os
import queue
import threading
import time

import cv2
import numpy as np
import onnxruntime

# folder of this script: the default "img" folder is next to it, wherever the script is started from
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))


def frame_preprocess(frame_in):
    """Camera frame (BGR) -> 28x28 float32 image in 0..1, white digit on black like MNIST."""
    gray = cv2.cvtColor(frame_in, cv2.COLOR_BGR2GRAY)

    # Crop only ROI (centered square)
    h, w = gray.shape
    size = min(h, w)
    x0, y0 = (w - size) // 2, (h - size) // 2
    gray = gray[y0:y0 + size, x0:x0 + size]

    # Thresholding to remove background: dark ink -> 0, keep the bright paper as-is
    _, mask = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)
    gray = cv2.bitwise_and(gray, gray, mask=mask)

    # Increase the contrast, then invert: black ink on white paper -> white digit on black
    gray = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)
    gray = cv2.bitwise_not(gray)

    # Resize to 28x28 as the input size of the ANN model, normalize to 0..1
    frame out = cv2.resize(gray, (28, 28), interpolation=cv2.INTER AREA)
def run_inference(session, x):
    """Run the model on one 28x28 image, return the predicted digit."""
    model_input = session.get_inputs()[0]
    # reshape the 28x28 image to the model input shape, e.g. (1, 28, 28) or (1, 784)
    # (a dynamic dimension such as "batch" is replaced by 1)
    shape = [d if isinstance(d, int) and d > 0 else 1 for d in model_input.shape]
    x = x.reshape(shape)

    output = session.run(None, {model_input.name: x})
    return int(np.argmax(output[0]))


def put_latest(q, item):
    """Put item into a size-1 queue, dropping the old frame if present."""
    try:
        q.put_nowait(item)
    except queue.Full:
        try:
            q.get_nowait()
        except queue.Empty:
            pass
        q.put_nowait(item)


def put_frame_to_queue(grab_frame, q, stop_event):
    """Producer thread: read frames from the camera and keep only the latest one in the queue."""
    try:
        while not stop_event.is_set():
            frame = grab_frame()
            if frame is None:
                print("camera returned no frame")
                break
            put_latest(q, frame)
    except Exception as e:
        if not stop_event.is_set():   # errors while closing the camera are expected
            print("camera error:", e)
    finally:
        stop_event.set()
```

```python
def open_camera(camera, usb_device, width, height):
    """Returns (grab_frame, close) functions for the Pi Camera or a USB webcam."""
    if camera == "picam":
        from libcamera import controls
        from picamera2 import Picamera2

        picam2 = Picamera2()
        # "RGB888" in Picamera2 is stored in B,G,R byte order -> already what OpenCV expects
        config = picam2.create_video_configuration(main={"size": (width, height), "format": "RGB888"})
        picam2.configure(config)
        # Camera Module 3 has autofocus, but it is off by default (fixed focus far away):
        # enable continuous autofocus so a digit held close to the camera is sharp
        if "AfMode" in picam2.camera_controls:
            picam2.set_controls({"AfMode": controls.AfModeEnum.Continuous})
        picam2.start()
        return (lambda: picam2.capture_array("main")), (lambda: (picam2.stop(), picam2.close()))

    # on the Pi 5 the camera index is not always 0, a /dev/v4l/by-id/... path is safer
    stream = cv2.VideoCapture(int(usb_device) if usb_device.isdigit() else usb_device)
    if not stream.isOpened():
        stream.release()
        raise RuntimeError(f"cannot open the USB camera {usb_device}")
    stream.set(cv2.CAP_PROP_FRAME_WIDTH, width)
    stream.set(cv2.CAP_PROP_FRAME_HEIGHT, height)

    def grab_frame():
        ret, frame = stream.read()
        return frame if ret else None

    return grab_frame, stream.release


def mode_image(infer, img_dir):
    """Inference with local images img/0.jpg ... 9.jpg
    infer(x) runs the model on one 28x28 image and returns the predicted digit."""
    for number in range(10):
        # read image
        img_path = os.path.join(img_dir, f"{number}.jpg")
        img = cv2.imread(img_path)
        if img is None:
            print("cannot read", img_path)
            continue

        # preprocess image and run inference
        x = frame_preprocess(img)
        t_start = time.perf_counter()
        predict = infer(x)
        t_infer = time.perf_counter() - t_start
        print(f"{img_path}: predicted number = {predict}  (inference {t_infer * 1000:.2f} ms)")

        # output / display
        cv2.putText(img, str(predict), (0, img.shape[0] - 40), cv2.FONT_HERSHEY_SIMPLEX,
                    10, (0, 255, 255), 4, cv2.LINE_AA)
        cv2.imshow("input number", img)
        cv2.imshow("model input", cv2.resize(x, (280, 280), interpolation=cv2.INTER_NEAREST))
        if cv2.waitKey(1500) & 0xFF == ord("q"):
            break
```

```
def mode_camera(infer, camera, usb_device, width, height, rotate=False):
    """Inference with the live camera, show the FPS until 'q' or Ctrl+C.
    infer(x) runs the model on one 28x28 image and returns the predicted digit.
    rotate=True turns the frame 90 degrees clockwise, for a camera mounted 90 degrees counter clockwise."""
    grab_frame, close_camera = open_camera(camera, usb_device, width, height)
    q = queue.Queue(maxsize=1)
    stop_event = threading.Event()
    t1 = threading.Thread(target=put_frame_to_queue, args=(grab_frame, q, stop_event), daemon=True)
    t1.start()

    fps = 0.0
    t_infer_avg = 0.0
    t_previous = time.perf_counter()
    t_print = t_previous
```

```
    try:
        while not stop_event.is_set():
            # wait for the most recent frame
            try:
                frame = q.get(timeout=0.5)
            except queue.Empty:
                continue

            # rotate back the frame of a camera mounted 90 degrees counter clockwise
            if rotate:
                frame = cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)

            # preprocess image and run inference
            x = frame_preprocess(frame)
            t_start = time.perf_counter()
            predict = infer(x)
            t_infer = time.perf_counter() - t_start

            # FPS measurement (wall clock, smoothed)
            t_current = time.perf_counter()
            fps = 0.9 * fps + 0.1 / (t_current - t_previous)
            t_infer_avg = 0.9 * t_infer_avg + 0.1 * t_infer
            t_previous = t_current
            if t_current - t_print >= 1.0:
                t_print = t_current
                print(f"FPS = {fps:5.1f} | inference {t_infer_avg * 1000:6.2f} ms | predicted number = {predict}")

            # output / display
            font = cv2.FONT_HERSHEY_SIMPLEX
            h, w, _ = frame.shape
            txcolor = (0, 255, 255)
            cv2.putText(frame, f"ONNX (CPU)  FPS={fps:.1f}", (10, 40), font, 1.2, txcolor, 3, cv2.LINE_AA)
            cv2.putText(frame, f"inference {t_infer_avg * 1000:.2f} ms", (10, 85), font, 1.2, txcolor, 3, cv2.LINE_AA)
            cv2.putText(frame, str(predict), (0, h - 40), font, 10, txcolor, 4, cv2.LINE_AA)
            size = min(h, w)
            cv2.rectangle(frame, ((w - size) // 2, (h - size) // 2), ((w + size) // 2, (h + size) // 2), txcolor, 2)
            cv2.imshow("input", frame)
            cv2.imshow("model input", cv2.resize(x, (280, 280), interpolation=cv2.INTER_NEAREST))

            # check keyboard for exit program
            if cv2.waitKey(1) & 0xFF == ord("q"):
                break
    finally:
        stop_event.set()
        t1.join(timeout=2.0)
        close_camera()
```

```python
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="MNIST inference with ONNX Runtime (CPU)")
    parser.add_argument("-e", "--model", help="ONNX model (.onnx)", default="models/model_mnist.onnx")
    parser.add_argument("-m", "--mode", help="test with local images or the live camera",
                        choices=["image", "camera"], default="image")
    parser.add_argument("-i", "--images", help="folder with 0.jpg ... 9.jpg (image mode)",
                        default=os.path.join(SCRIPT_DIR, "img"))
    parser.add_argument("-c", "--camera", help="Pi Camera Module 3 or USB webcam (camera mode)",
                        choices=["picam", "usb"], default="picam")
    parser.add_argument("-u", "--usb-device", help="USB webcam: index or /dev/v4l/by-id/... path",
                        default="0")
    parser.add_argument("-r", "--rotate", action="store_true",
                        help="rotate the camera frame 90 deg clockwise (camera mounted 90 deg counter clockwise)")
    parser.add_argument("--width", type=int, default=1280)
    parser.add_argument("--height", type=int, default=720)
    args = parser.parse_args()

    # Load the ONNX model
    session = onnxruntime.InferenceSession(args.model, providers=["CPUExecutionProvider"])
    model_input = session.get_inputs()[0]
    print("input :", model_input.name, model_input.shape)
    print("output:", session.get_outputs()[0].name, session.get_outputs()[0].shape)

    def infer(x):
        return run_inference(session, x)

    try:
        if args.mode == "image":
            mode_image(infer, args.images)
        else:
            mode_camera(infer, args.camera, args.usb_device, args.width, args.height, args.rotate)
    except KeyboardInterrupt:
        pass
    finally:
        cv2.destroyAllWindows()
        print("inference is over")
```

Experiment 3. Deploy the model in RPi 5 using Hailo-8 NPU

(Follow the Lab Slide)

Python script for all Hailo compilation process:

- onnx2har.py

- optimize.py

- compile.py

onnx2har.py (Parsing)

```python
from hailo_sdk_client import ClientRunner
import argparse
import json



parser = argparse.ArgumentParser()
# Adding optional argument
parser.add_argument("-c", "--cfg", help="ONNX 2 HAR configuration file as .json file", default='.json')
args = parser.parse_args()
f = open(args.cfg)
data = json.load(f)

quantized_har_path = data["output_path"] + data["quantized_har_name"]
runner = ClientRunner(har=quantized_har_path)
hef = runner.compile()
file_name = data["output_path"] + data["model_name"] + ".hef"
with open(file_name, "wb") as f:
    f.write(hef)
```

optimize.py

```
from hailo_sdk_client import ClientRunner
import argparse
import json



parser = argparse.ArgumentParser()
# Adding optional argument
parser.add_argument("-c", "--cfg", help="ONNX 2 HAR configuration file as .json file", default='.json')
args = parser.parse_args()
f = open(args.cfg)
data = json.load(f)

model_name = data['model_name']
hailo_model_har_name = data['output_path'] + data['har_name']
runner = ClientRunner(har=hailo_model_har_name)

# Normalization Section
# Now we will create a model script, that tells the compiler to add a normalization on the beginning
# of the model (that is why we didn't normalize the calibration set;
# Otherwise we would have to normalize it before using it)
# Batch size is 8 by default
#alls = "normalization1 = normalization([123.675, 116.28, 103.53], [58.395, 57.12, 57.375])\n"
#runner.load_model_script(alls)

# Model script file from the config (e.g. yolov5n.alls: normalization + NMS on the chip)
if "model_script" in data:
    runner.load_model_script(data["model_script"])

# Call Optimize to perform the optimization process
runner.optimize(data['calib_set'])
# Save the result state to a Quantized HAR file
quantized_model_har_path = data['output_path'] + data['quantized_har_name']
runner.save_har(quantized_model_har_path)
```

compile.py

```
from hailo_sdk_client import ClientRunner
import argparse
import json



parser = argparse.ArgumentParser()
# Adding optional argument
parser.add_argument("-c", "--cfg", help="ONNX 2 HAR configuration file as .json file", default='.json')
args = parser.parse_args()
f = open(args.cfg)
data = json.load(f)

quantized_har_path = data["output_path"] + data["quantized_har_name"]
runner = ClientRunner(har=quantized_har_path)
hef = runner.compile()
file_name = data["output_path"] + data["model_name"] + ".hef"
with open(file_name, "wb") as f:
    f.write(hef)
```

inference_mnist_hailo.py

```
"""
inference_mnist_hailo.py

Run the MNIST digit classifier (DIY ANN) on the Hailo-8 NPU with HailoRT.
The model is compiled to .hef with the Hailo Dataflow Compiler.

    python3 inference_mnist_hailo.py --mode image                    # test images img/0.jpg ... 9.jpg
    python3 inference_mnist_hailo.py --mode camera                   # Pi Camera Module 3, live FPS
    python3 inference_mnist_hailo.py --mode camera --camera usb      # USB webcam
    python3 inference_mnist_hailo.py --mode camera -r                # camera mounted 90 deg counter clockwise

Press 'q' in the video window or Ctrl+C to quit.
Compare with inference_mnist_onnx.py (same program on the Raspberry Pi 5 CPU):
the "inference ... ms" value is the speed of the model itself. The FPS is the speed of the whole loop
and is limited by the camera (30 FPS) and the display, so it is about the same for both.
"""
import argparse
import os
import queue
import threading
import time

import cv2
import numpy as np
from hailo_platform import (
    HEF,
    ConfigureParams,
    FormatType,
    HailoSchedulingAlgorithm,
    HailoStreamInterface,
    InferVStreams,
    InputVStreamParams,
    OutputVStreamParams,
    VDevice,
)

# folder of this script: the default "img" folder is next to it, wherever the script is started from
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))


def frame_preprocess(frame_in):
    """Camera frame (BGR) -> 28x28 float32 image in 0..1, white digit on black like MNIST."""
    gray = cv2.cvtColor(frame_in, cv2.COLOR_BGR2GRAY)

    # Crop only ROI (centered square)
    h, w = gray.shape
    size = min(h, w)
    x0, y0 = (w - size) // 2, (h - size) // 2
    gray = gray[y0:y0 + size, x0:x0 + size]

    # Thresholding to remove background: dark ink -> 0, keep the bright paper as-is
    _, mask = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)
    gray = cv2.bitwise_and(gray, gray, mask=mask)

    # Increase the contrast, then invert: black ink on white paper -> white digit on black
    gray = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)
    gray = cv2.bitwise_not(gray)

    # Resize to 28x28 as the input size of the ANN model, normalize to 0..1
    frame_out = cv2.resize(gray, (28, 28), interpolation=cv2.INTER_AREA)
    return frame_out.astype(np.float32) / 255.0


def run_inference(infer_pipeline, input_vstream_info, output_vstream_info, x):
    """Run the model on one 28x28 image, return the predicted digit."""
    # reshape the 28x28 image to the model input shape + batch dimension, e.g. (1, 784)
    x = x.reshape((1,) + tuple(input_vstream_info.shape))

    infer_results = infer_pipeline.infer({input_vstream_info.name: x})
    # The result output tensor is infer_results[output_vstream_info.name]
    return int(np.argmax(infer_results[output_vstream_info.name]))


def put_latest(q, item):
    """Put item into a size-1 queue, dropping the old frame if present."""
    try:
        q.put_nowait(item)
    except queue.Full:
        try:
            q.get_nowait()
        except queue.Empty:
            pass
        q.put_nowait(item)
```

```python
def put_frame_to_queue(grab_frame, q, stop_event):
    """Producer thread: read frames from the camera and keep only the latest one in the queue."""
    try:
        while not stop_event.is_set():
            frame = grab_frame()
            if frame is None:
                print("camera returned no frame")
                break
            put_latest(q, frame)
    except Exception as e:
        if not stop_event.is_set():   # errors while closing the camera are expected
            print("camera error:", e)
    finally:
        stop_event.set()


def open_camera(camera, usb_device, width, height):
    """Returns (grab_frame, close) functions for the Pi Camera or a USB webcam."""
    if camera == "picam":
        from libcamera import controls
        from picamera2 import Picamera2

        picam2 = Picamera2()
        # "RGB888" in Picamera2 is stored in B,G,R byte order -> already what OpenCV expects
        config = picam2.create_video_configuration(main={"size": (width, height), "format": "RGB888"})
        picam2.configure(config)
        # Camera Module 3 has autofocus, but it is off by default (fixed focus far away):
        # enable continuous autofocus so a digit held close to the camera is sharp
        if "AfMode" in picam2.camera_controls:
            picam2.set_controls({"AfMode": controls.AfModeEnum.Continuous})
        picam2.start()
        return (lambda: picam2.capture_array("main")), (lambda: (picam2.stop(), picam2.close()))

    # on the Pi 5 the camera index is not always 0, a /dev/v4l/by-id/... path is safer
    stream = cv2.VideoCapture(int(usb_device) if usb_device.isdigit() else usb_device)
    if not stream.isOpened():
        stream.release()
        raise RuntimeError(f"cannot open the USB camera {usb_device}")
    stream.set(cv2.CAP_PROP_FRAME_WIDTH, width)
    stream.set(cv2.CAP_PROP_FRAME_HEIGHT, height)

    def grab_frame():
        ret, frame = stream.read()
        return frame if ret else None

    return grab_frame, stream.release
```

```python
def mode_image(infer, img_dir):
    """Inference with local images img/0.jpg ... 9.jpg
    infer(x) runs the model on one 28x28 image and returns the predicted digit."""
    for number in range(10):
        # read image
        img_path = os.path.join(img_dir, f"{number}.jpg")
        img = cv2.imread(img_path)
        if img is None:
            print("cannot read", img_path)
            continue

        # preprocess image and run inference
        x = frame_preprocess(img)
        t_start = time.perf_counter()
        predict = infer(x)
        t_infer = time.perf_counter() - t_start
        print(f"{img_path}: predicted number = {predict}  (inference {t_infer * 1000:.2f} ms)")

        # output / display
        cv2.putText(img, str(predict), (0, img.shape[0] - 40), cv2.FONT_HERSHEY_SIMPLEX,
                    10, (0, 255, 255), 4, cv2.LINE_AA)
        cv2.imshow("input number", img)
        cv2.imshow("model input", cv2.resize(x, (280, 280), interpolation=cv2.INTER_NEAREST))
        if cv2.waitKey(1500) & 0xFF == ord("q"):
            break
```

```python
def mode_camera(infer, camera, usb_device, width, height, rotate=False):
    """Inference with the live camera, show the FPS until 'q' or Ctrl+C.
    infer(x) runs the model on one 28x28 image and returns the predicted digit.
    rotate=True turns the frame 90 degrees clockwise, for a camera mounted 90 degrees counter clockwise."""
    grab_frame, close_camera = open_camera(camera, usb_device, width, height)
    q = queue.Queue(maxsize=1)
    stop_event = threading.Event()
    t1 = threading.Thread(target=put_frame_to_queue, args=(grab_frame, q, stop_event), daemon=True)
    t1.start()

    fps = 0.0
    t_infer_avg = 0.0
    t_previous = time.perf_counter()
    t_print = t_previous
    try:
        while not stop_event.is_set():
            # wait for the most recent frame
            try:
                frame = q.get(timeout=0.5)
            except queue.Empty:
                continue

            # rotate back the frame of a camera mounted 90 degrees counter clockwise
            if rotate:
                frame = cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)

            # preprocess image and run inference
            x = frame_preprocess(frame)
            t_start = time.perf_counter()
            predict = infer(x)
            t_infer = time.perf_counter() - t_start

            # FPS measurement (wall clock, smoothed)
            t_current = time.perf_counter()
            fps = 0.9 * fps + 0.1 / (t_current - t_previous)
            t_infer_avg = 0.9 * t_infer_avg + 0.1 * t_infer
            t_previous = t_current
            if t_current - t_print >= 1.0:
                t_print = t_current
                print(f"FPS = {fps:5.1f} | inference {t_infer_avg * 1000:6.2f} ms | predicted number = {predict}")

            # output / display
            font = cv2.FONT_HERSHEY_SIMPLEX
            h, w, _ = frame.shape
            txcolor = (0, 255, 255)
            cv2.putText(frame, f"Hailo-8 (NPU)  FPS={fps:.1f}", (10, 40), font, 1.2, txcolor, 3, cv2.LINE_AA)
            cv2.putText(frame, f"inference {t_infer_avg * 1000:.2f} ms", (10, 85), font, 1.2, txcolor, 3, cv2.LINE_AA)
            cv2.putText(frame, str(predict), (0, h - 40), font, 10, txcolor, 4, cv2.LINE_AA)
            size = min(h, w)
            cv2.rectangle(frame, ((w - size) // 2, (h - size) // 2), ((w + size) // 2, (h + size) // 2), txcolor, 2)
            cv2.imshow("input", frame)
            cv2.imshow("model input", cv2.resize(x, (280, 280), interpolation=cv2.INTER_NEAREST))

            # check keyboard for exit program
            if cv2.waitKey(1) & 0xFF == ord("q"):
                break


    finally:
        stop_event.set()
        t1.join(timeout=2.0)
        close_camera()
```

```python
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="MNIST inference on the Hailo-8 NPU")
    parser.add_argument("-e", "--model", help="Hailo Executable File (.hef)", default="models/model_mnist.hef")
    parser.add_argument("-m", "--mode", help="test with local images or the live camera",
                        choices=["image", "camera"], default="image")
    parser.add_argument("-i", "--images", help="folder with 0.jpg ... 9.jpg (image mode)",
                        default=os.path.join(SCRIPT_DIR, "img"))
    parser.add_argument("-c", "--camera", help="Pi Camera Module 3 or USB webcam (camera mode)",
                        choices=["picam", "usb"], default="picam")
    parser.add_argument("-u", "--usb-device", help="USB webcam: index or /dev/v4l/by-id/... path",
                        default="0")
    parser.add_argument("-r", "--rotate", action="store_true",
                        help="rotate the camera frame 90 deg clockwise (camera mounted 90 deg counter clockwise)")
    parser.add_argument("--width", type=int, default=1280)
    parser.add_argument("--height", type=int, default=720)
    args = parser.parse_args()

    # Disable the model scheduler: the network group is activated manually below.
    # Only one program can use the Hailo-8 at a time.
    params = VDevice.create_params()
    params.scheduling_algorithm = HailoSchedulingAlgorithm.NONE

    # The "with" statements release the NPU when the program ends, also after Ctrl+C or an error
    with VDevice(params=params) as target:
        # Loading compiled HEF to device
        hef = HEF(args.model)
        configure_params = ConfigureParams.create_from_hef(hef=hef, interface=HailoStreamInterface.PCIe)
        network_group = target.configure(hef, configure_params)[0]
        network_group_params = network_group.create_params()

        # Create input and output virtual streams params
        # FLOAT32 input : HailoRT quantizes the 0..1 image on the host
        # FLOAT32 output: HailoRT de-quantizes the scores
        input_vstreams_params = InputVStreamParams.make(network_group, format_type=FormatType.FLOAT32)
        output_vstreams_params = OutputVStreamParams.make(network_group, format_type=FormatType.FLOAT32)

        input_vstream_info = hef.get_input_vstream_infos()[0]
        output_vstream_info = hef.get_output_vstream_infos()[0]
        print("input :", input_vstream_info.name, input_vstream_info.shape)
        print("output:", output_vstream_info.name, output_vstream_info.shape)

        # Open the inference pipeline and activate the network group ONCE, for all frames
        with InferVStreams(network_group, input_vstreams_params, output_vstreams_params) as infer_pipeline, \
                network_group.activate(network_group_params):

            def infer(x):
                return run_inference(infer_pipeline, input_vstream_info, output_vstream_info, x)

            try:
                if args.mode == "image":
                    mode_image(infer, args.images)
                else:
                    mode_camera(infer, args.camera, args.usb_device, args.width, args.height, args.rotate)
            except KeyboardInterrupt:
                pass
            finally:
                cv2.destroyAllWindows()
                print("inference is over")
```